

Deep Learning

UNIT – 1: Basics of linear algebra for deep learning

Deep Learning: Deep learning is a subset of machine learning that uses multilayered neural networks, called deep neural networks, to simulate the complex decision-making power of the human brain. Some form of deep learning powers most of the artificial intelligence (AI) applications in our lives today.

The chief difference between deep learning and machine learning is the structure of the underlying neural network architecture. “Nondeep,” traditional machine learning models use simple neural networks with one or two computational layers. Deep learning models use three or more layers, but typically hundreds or thousands of layers to train the models.

While supervised learning models require structured, labeled input data to make accurate outputs, deep learning models can use unsupervised learning. With unsupervised learning, deep learning models can extract the characteristics, features and relationships they need to make accurate outputs from raw, unstructured data. Additionally, these models can even evaluate and refine their outputs for increased precision.

Deep learning is an aspect of data science that drives many applications and services that improve automation, performing analytical and physical tasks without human intervention. This enables many everyday products and services, such as digital assistants, voice-enabled TV remotes, credit card fraud detection, self-driving cars and generative AI.

Linear Algebra:

Linear algebra is a branch of mathematics that is widely used throughout science and engineering. Yet because linear algebra is a form of continuous rather than discrete mathematics, many computer scientists have little experience with it. A good understanding of linear algebra is essential for understanding and working with many machine learning algorithms, especially deep learning algorithms.

Vectors: A vector is an array of numbers. The numbers are arranged in order. We can identify each individual number by its index in that ordering. Typically we give vectors lowercase names in bold typeface, such as $\mathbf{x}$. The elements of the vector are identified by writing its name in italic typeface, with a subscript. The first element of $\mathbf{x}$ is x_1 , the second element is x_2 , and so on. We also need to say what kind of numbers are stored in the vector. If each element is in $\mathbb{R}$, and the vector has n elements, then the vector lies in the set formed by taking the Cartesian product of $\mathbb{R}$ n times, denoted as $\mathbb{R}^n$. When we need to explicitly identify the elements of a vector, we write them as a column enclosed in square brackets:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}. \quad (2.1)$$

We can think of vectors as identifying points in space, with each element giving the coordinate along a different axis.

Matrices: A matrix is a 2-D array of numbers, so each element is identified by two indices instead of just one. We usually give matrices uppercase variable names with bold typeface, such as $\mathbf{A}$. If a real-valued matrix $\mathbf{A}$ has a height of m and a width of n , then we say that $\mathbf{A} \in \mathbb{R}^{m \times n}$. We usually identify the elements of a matrix using its name in italic but not bold font, and the indices are listed with separating commas. For example, $A_{1,1}$ is the upper left entry of $\mathbf{A}$ and $A_{m,n}$ is the bottom right entry. We can identify all the numbers with vertical coordinate i by writing a “.” for the horizontal coordinate. For example, $\mathbf{A}_{i,:}$ denotes the horizontal cross section of $\mathbf{A}$ with vertical coordinate i . This is known as the i -th **row** of $\mathbf{A}$. Likewise, $\mathbf{A}_{:,i}$ is the i -th **column** of $\mathbf{A}$. When we need to explicitly identify the elements of a matrix, we write them as an array enclosed in square brackets:

$$\begin{bmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{bmatrix}. \quad (2.2)$$

Sometimes we may need to index matrix-valued expressions that are not just a single letter. In this case, we use subscripts after the expression but do not convert anything to lowercase. For example, $f(\mathbf{A})_{i,j}$ gives element (i, j) of the matrix computed by applying the function f to $\mathbf{A}$.

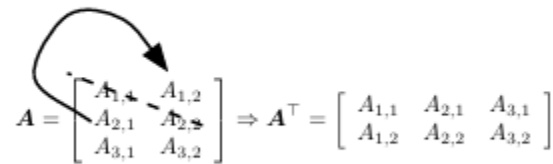
Tensors: In some cases we will need an array with more than two axes. In the general case, an array of numbers arranged on a regular grid with a variable number of axes is known as a tensor. We denote a tensor named “A” with this typeface: **A**. We identify the element of **A** at coordinates (i, j, k) by writing $A_{i,j,k}$.

$$(\mathbf{A}^\top)_{i,j} = A_{j,i}. \quad (2.3)$$

Vectors can be thought of as matrices that contain only one column. The transpose of a vector is therefore a matrix with only one row. Sometimes we define a vector by writing out its elements in the text inline as a row matrix, then using the transpose operator to turn it into a standard column vector, for example $\mathbf{x} = [x_1, x_2, x_3]^\top$.

Scalar

A scalar can be thought of as a matrix with only a single entry. From this, we can see that a scalar is its own transpose: $a = a^\top$.



$$\mathbf{A} = \begin{bmatrix} A_{1,1} & A_{1,2} & A_{1,3} \\ A_{2,1} & A_{2,2} & A_{2,3} \\ A_{3,1} & A_{3,2} & A_{3,3} \end{bmatrix} \Rightarrow \mathbf{A}^\top = \begin{bmatrix} A_{1,1} & A_{2,1} & A_{3,1} \\ A_{1,2} & A_{2,2} & A_{3,2} \\ A_{1,3} & A_{2,3} & A_{3,3} \end{bmatrix}$$

Figure 2.1: The transpose of the matrix can be thought of as a mirror image across the main diagonal.

We can add matrices to each other, as long as they have the same shape, just by adding their corresponding elements: $\mathbf{C} = \mathbf{A} + \mathbf{B}$ where $C_{i,j} = A_{i,j} + B_{i,j}$.

MULTIPLICATION OF MATRICES

One of the most important operations involving matrices is multiplication of two matrices. The **matrix product** of matrices **A** and **B** is a third matrix **C**. In order for this product to be defined, **A** must have the same number of columns as **B** has rows. If **A** is of shape $m \times n$ and **B** is of shape $n \times p$, then **C** is of shape $m \times p$. We can write the matrix product just by placing two or more matrices together, for example,

$$\mathbf{C} = \mathbf{AB}. \quad (2.4)$$

The product operation is defined by

$$C_{i,j} = \sum_k A_{i,k} B_{k,j}. \quad (2.5)$$

Note that the standard product of two matrices is *not* just a matrix containing the product of the individual elements. Such an operation exists and is called the **element-wise product**, or **Hadamard product**, and is denoted as $\mathbf{A} \odot \mathbf{B}$.

Matrix product operations have many useful properties that make mathematical analysis of matrices more convenient. For example, matrix multiplication is distributive:

$$\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC}. \quad (2.6)$$

It is also associative:

$$\mathbf{A}(\mathbf{BC}) = (\mathbf{AB})\mathbf{C}. \quad (2.7)$$

Matrix multiplication is *not* commutative (the condition $\mathbf{AB} = \mathbf{BA}$ does not always hold), unlike scalar multiplication. However, the dot product between two vectors is commutative:

$$\mathbf{x}^\top \mathbf{y} = \mathbf{y}^\top \mathbf{x}. \quad (2.8)$$

The transpose of a matrix product has a simple form:

$$(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top. \quad (2.9)$$

2.3 Identity and Inverse Matrices

Linear algebra offers a powerful tool called **matrix inversion** that enables us to analytically solve equation 2.11 for many values of $\mathbf{A}$.

To describe matrix inversion, we first need to define the concept of an **identity matrix**. An identity matrix is a matrix that does not change any vector when we multiply that vector by that matrix. We denote the identity matrix that preserves n -dimensional vectors as $\mathbf{I}_n$. Formally, $\mathbf{I}_n \in \mathbb{R}^{n \times n}$, and

$$\forall \mathbf{x} \in \mathbb{R}^n, \mathbf{I}_n \mathbf{x} = \mathbf{x}. \quad (2.20)$$

The structure of the identity matrix is simple: all the entries along the main diagonal are 1, while all the other entries are zero. See figure 2.2 for an example.

The **matrix inverse** of $\mathbf{A}$ is denoted as $\mathbf{A}^{-1}$, and it is defined as the matrix such that

$$\mathbf{A}^{-1} \mathbf{A} = \mathbf{I}_n. \quad (2.21)$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Figure 2.2: Example identity matrix: This is $\mathbf{I}_3$.

Eigen values and Eigen vectors

Eigen values and Eigen vectors are the scalar and vector quantities associated with matrices used for linear transformations. The vector that only changes by a scalar factor after applying a transformation is called an eigenvector, and the scalar value attached to the eigenvector is called the eigen value.

Eigenvalues are scalar values associated with a square matrix that measure how a matrix transforms a vector. If a matrix A multiplies a vector v , and the result is a scalar multiple of v , then that scalar is the eigenvalue corresponding to the eigenvector v . Eigenvalues are widely used in fields like physics, engineering, and data science.

Hence, these characteristic values indicate the factor by which eigenvectors are stretched in their direction. It doesn't involve the change in the direction of the vector except when the eigenvalue is negative. When the eigenvalue is negative the direction is just reversed.

The equation for eigenvalue is given by $Av = \lambda v$

Where, A is the matrix, v is associated eigenvector, and λ is scalar eigenvalue.

What are Eigenvectors?

Eigenvectors for square matrices are defined as non-zero vector values which when multiplied by the square matrices give the scalar multiple of the vector, i.e. we define an eigenvector for matrix A to be " v " if it specifies the condition, $Av = \lambda v$

The scalar multiple λ in the above case is called the eigenvalue of the square matrix. We always have to find the eigenvalues of the square matrix first before finding the eigenvectors of the matrix.

Eigenvector Equation

The Eigenvector equation is the equation that is used to find the eigenvector of any square matrix. The eigenvector equation is,

$$Av = \lambda v$$

Where,

- A is the given square matrix,
- v is the eigenvector of matrix A , and
- λ is any scalar multiple.

Principal Component Analysis

PCA (Principal Component Analysis) is a dimensionality reduction technique used in data analysis and machine learning. **It helps you to reduce the number of features in a dataset while keeping the most important information.** It changes your original features into new features these new features don't overlap with each other and the first few keep most of the important differences found in the original data.

PCA is commonly used for data preprocessing for use with machine learning algorithms. It helps to remove redundancy, improve computational efficiency and make data easier to visualize and analyze especially when dealing with high-dimensional data.

How Principal Component Analysis Works

PCA uses linear algebra to transform data into new features called principal components. It finds these by calculating eigenvectors (directions) and eigenvalues (importance - magnitude) from the covariance matrix. PCA selects the top components with the highest eigenvalues and projects the data onto them simplify the dataset.

***Note:** It prioritizes the directions where the data varies the most because more variation = more useful information.*

Imagine you're looking at a messy cloud of data points like stars in the sky and want to simplify it. PCA helps you find the "most important angles" to view this cloud so you don't miss the big patterns. Here's how it works step by step:

Step 1: Standardize the Data

Different features may have different units and scales like salary vs. age. To compare them fairly PCA first standardizes the data by making each feature have:

- A mean of 0
- A standard deviation of 1

$$Z = \frac{X - \mu}{\sigma}$$

where:

- μ is the mean of independent features $\mu = \{\mu_1, \mu_2, \dots, \mu_m\}$
- σ is the standard deviation of independent features $\sigma = \{\sigma_1, \sigma_2, \dots, \sigma_m\}$

Step 2: Calculate Covariance Matrix

Next PCA calculates the covariance matrix to see how features relate to each other whether they increase or decrease together. The covariance between two features x_1 and x_2 is:

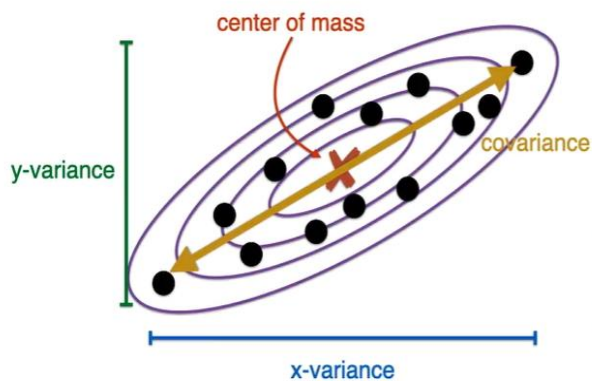
$$\text{cov}(x_1, x_2) = \frac{\sum_{i=1}^n (x_{1i} - \bar{x}_1)(x_{2i} - \bar{x}_2)}{n-1}$$

Where:

- $\bar{x}_1$ and $\bar{x}_2$ are the mean values of features x_1 and x_2
- n is the number of data points

The value of covariance can be positive, negative or zeros.

The covariance matrix

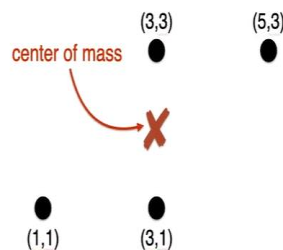


$$\mu = \text{Average}$$

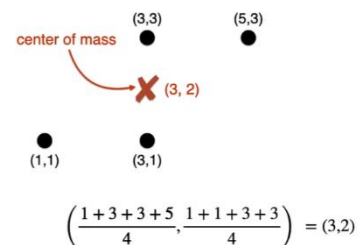
$$\Sigma = \begin{pmatrix} \text{Var}(x) & \text{Cov}(x, y) \\ \text{Cov}(x, y) & \text{Var}(y) \end{pmatrix}$$

Step 2: Calculate
Covariance
Matrix

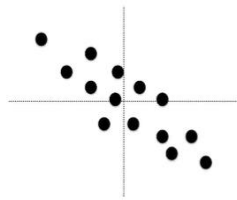
Average



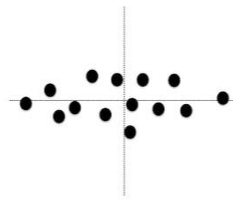
Average



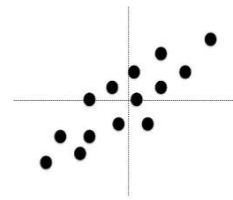
In general



Negative covariance

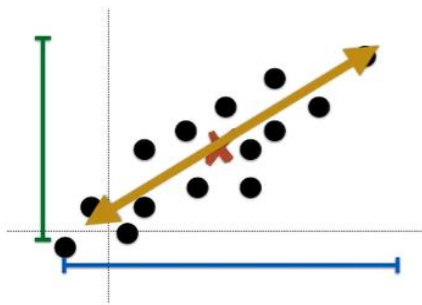


Zero covariance
(or very small)



Positive covariance

Formulas



Points

(x_1, y_1)
 (x_2, y_2)
 $\vdots$
 (x_n, y_n)

Mean

$$(\mu_x, \mu_y) = \left(\frac{1}{n} \sum_{i=1}^n x_i, \frac{1}{n} \sum_{i=1}^n y_i \right)$$

x-Variance

$$\text{var}(x) = \frac{1}{n} \sum_{i=1}^n (x_i - \mu_x)^2$$

y-Variance

$$\text{var}(y) = \frac{1}{n} \sum_{i=1}^n (y_i - \mu_y)^2$$

Covariance

$$\text{cov}(x, y) = \frac{1}{n} \sum_{i=1}^n (x_i - \mu_x)(y_i - \mu_y)$$

Covariance matrix

$$\Sigma = \begin{pmatrix} \text{var}(x) & \text{cov}(x, y) \\ \text{cov}(x, y) & \text{var}(y) \end{pmatrix}$$

Follow the link for the Covariance Matrix
<https://www.youtube.com/watch?v=WBlnwvjfMtQ&t=495s>

Step 3: Find the Principal Components

PCA identifies **new axes** where the data spreads out the most:

- **1st Principal Component (PC1):** The direction of maximum variance (most spread).
- **2nd Principal Component (PC2):** The next best direction, *perpendicular to PC1* and so on.

These directions come from the **eigenvectors** of the covariance matrix and their importance is measured by **eigenvalues**. For a square matrix A an **eigenvector** X (a non-zero vector) and its corresponding **eigenvalue** λ satisfy:

$$AX = \lambda X$$

This means:

- When A acts on X it only stretches or shrinks X by the scalar λ .
- The direction of X remains unchanged hence eigenvectors define "stable directions" of A .

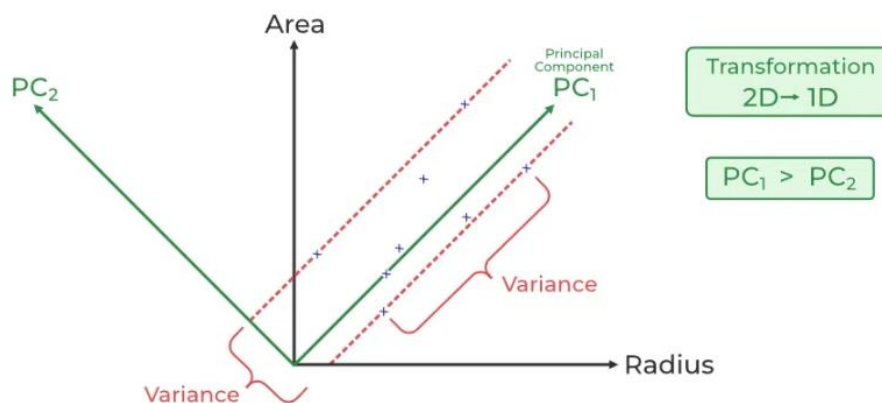
Eigenvalues help rank these directions by importance.

Step 4: Pick the Top Directions & Transform Data

After calculating the eigenvalues and eigenvectors PCA ranks them by the amount of information they capture. We then:

1. **Select the top k components** that capture most of the variance like 95%.
2. **Transform the original dataset** by projecting it onto these top components.

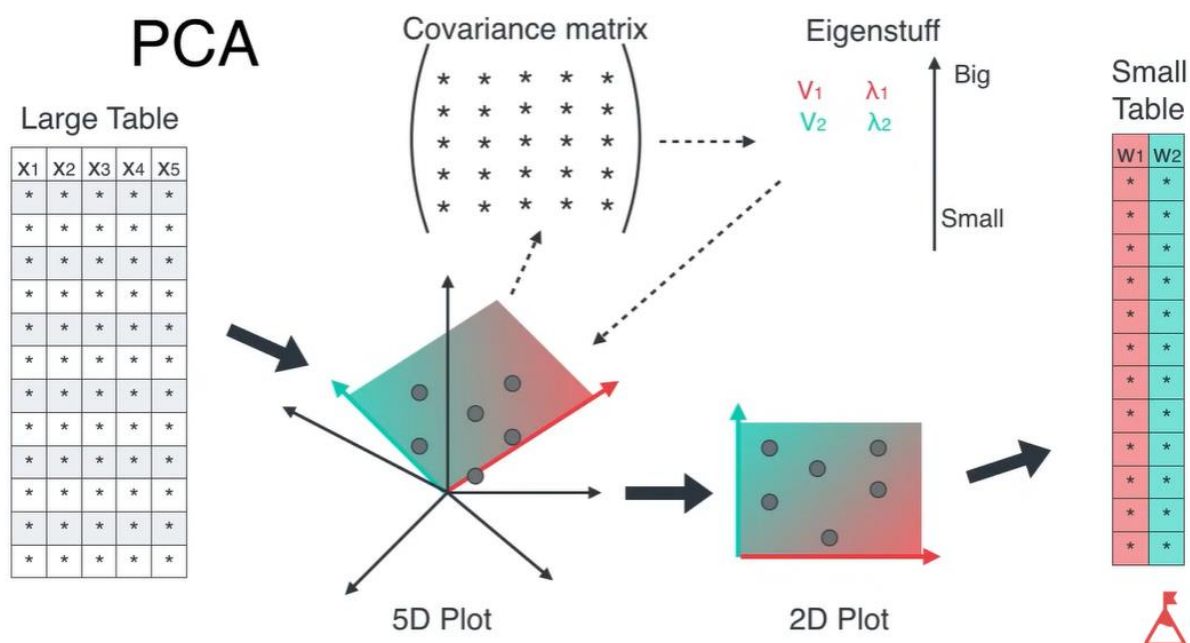
This means we reduce the number of features (dimensions) while keeping the important patterns in the data.



Transform this 2D dataset into a 1D representation while preserving as much variance as possible.

In the above image the original dataset has two features "Radius" and "Area" represented by the black axes. PCA identifies two new directions: **PC₁** and **PC₂** which are the **principal components**.

- These new axes are rotated versions of the original ones. **PC₁** captures the maximum variance in the data meaning it holds the most information while **PC₂** captures the remaining variance and is perpendicular to PC₁.
- The spread of data is much wider along PC₁ than along PC₂. This is why PC₁ is chosen for dimensionality reduction. By projecting the data points (blue crosses) onto PC₁ we effectively **transform the 2D data into 1D** and retain most of the important structure and patterns.



Follow the link for Principal Component Analysis

<https://www.youtube.com/watch?v=g-Hb26agBFg>

IMAGE VECTORIZATION

Image vectorization is the process of converting raster images (made of pixels) into vector images (made of paths), allowing for scalable graphics without loss of quality.

What is Image Vectorization?

Image vectorization transforms raster images, which consist of a grid of pixels, into vector images that are defined by mathematical equations. This process allows the resulting vector images to be resized without losing quality, making them ideal for various applications such as logos, illustrations, and technical drawings.

How Does It Work?

The vectorization process typically involves the following steps:

1. **Edge Detection:** The software analyzes the raster image to identify edges and areas of similar color or brightness.
2. **Shape Creation:** Based on the detected edges, the software creates geometric shapes (lines, curves, and polygons) that represent the image.
3. **Output Generation:** The final vector image can be saved in various formats such as SVG, EPS, or PDF, which are suitable for different applications.

Benefits of Vector Images

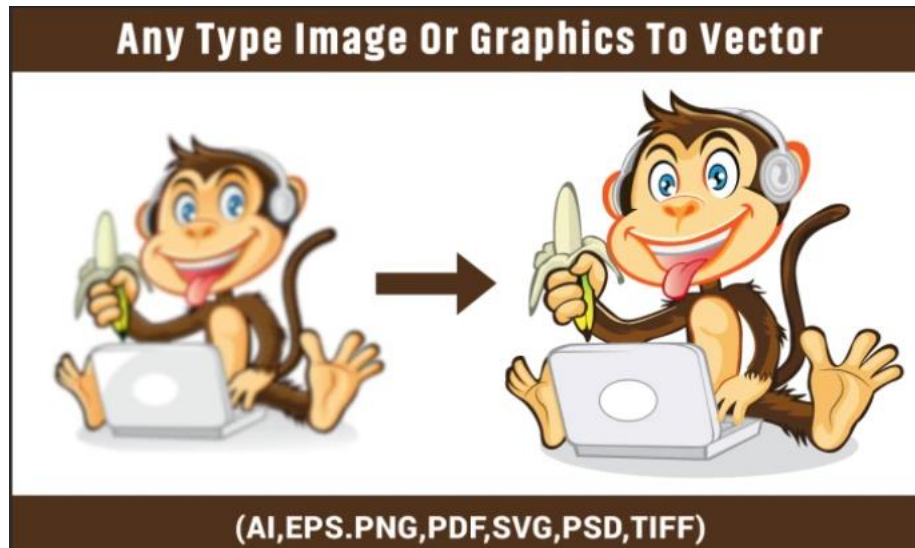
- **Scalability:** Vector images can be resized to any dimension without losing clarity or quality, making them perfect for print and digital media.
- **Editability:** Vector graphics can be easily edited and manipulated, allowing for greater flexibility in design.
- **Smaller File Sizes:** Vector files are often smaller than their raster counterparts, which can help save storage space and improve loading times on websites.

Tools for Image Vectorization

Several online tools and software can help you vectorize images, including:

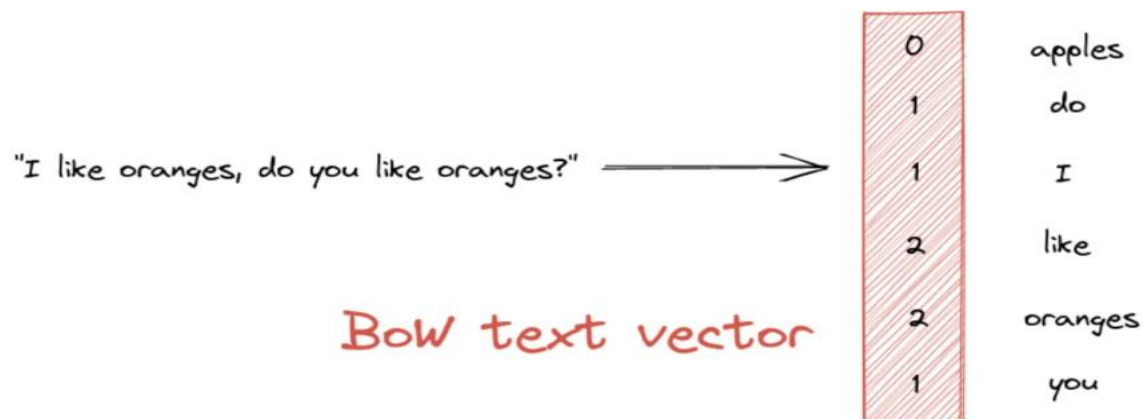
- **Vectorizer:** A free online tool that allows users to upload images and convert them to vector format without registration.
- **Vector Magic:** This tool provides automatic conversion of bitmap images to vector formats, offering a user-friendly interface.
- **Galaxy AI:** An AI-based vectorization tool that converts images quickly and easily, supporting various file formats.

By utilizing these tools, you can easily convert your raster images into high-quality vector graphics suitable for various design projects.



Text Vectorization

converts text into a vector representing the frequency of words, disregarding grammar and word order. It counts the occurrences of each word in a document and generates a vector based on these counts.



Advantages of Bag of Words (BoW)

- Simple and easy to implement.
- Provides a clear and interpretable representation of text.

Disadvantages of Bag of Words (BoW)

- Ignores the order and context of words.
- Results in high-dimensional and sparse matrices.
- Fails to capture semantic meaning and relationships between words.

```
from sklearn.feature_extraction.text import CountVectorizer

documents = [
    "The cat sat on the mat.",
    "The dog sat on the log.",
    "Cats and dogs are pets."
]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(documents)

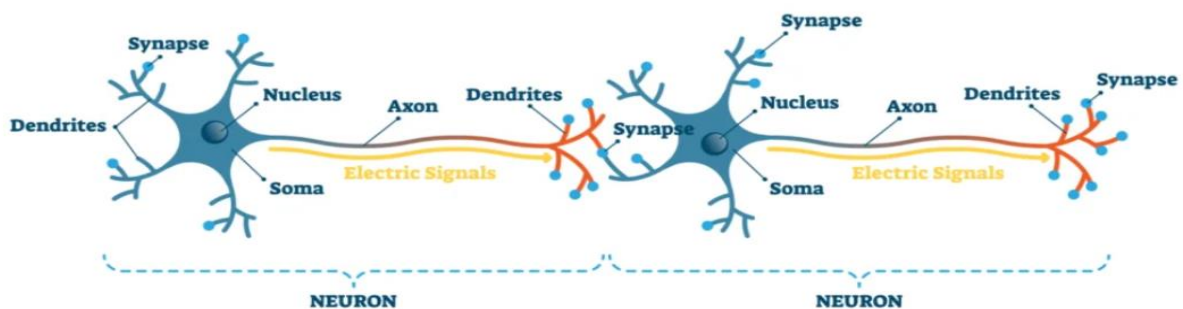
print(X.toarray())
print(vectorizer.get_feature_names_out())
```

Output:

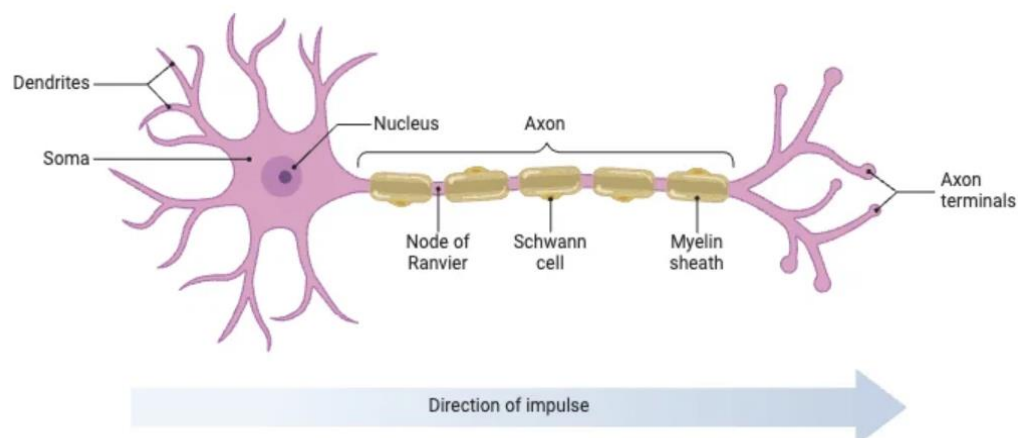
```
[[0 0 1 0 0 0 0 1 1 0 1 2]
 [0 0 0 0 1 0 1 0 1 0 1 2]
 [1 1 0 1 0 1 0 0 0 1 0 0]]
['and' 'are' 'cat' 'cats' 'dog' 'dogs' 'log' 'mat' 'on' 'pets' 'sat' 'the']
```

Bag of Words (BoW)

BIOLOGICAL NEURON

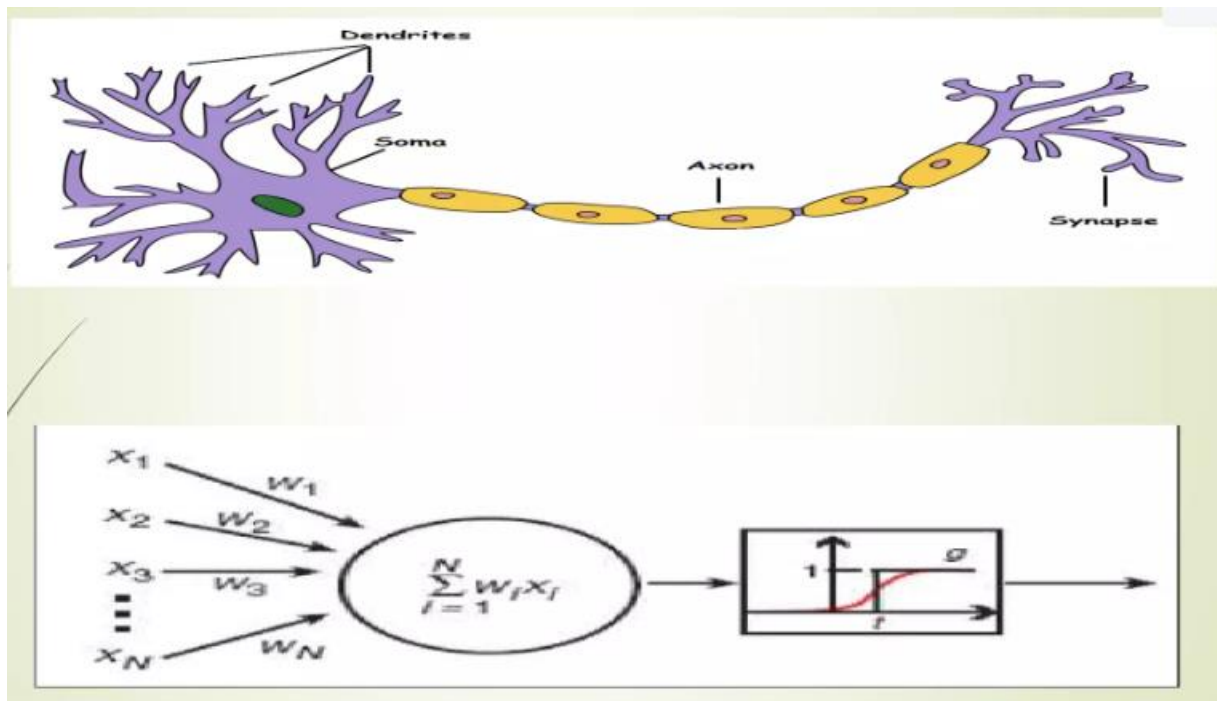
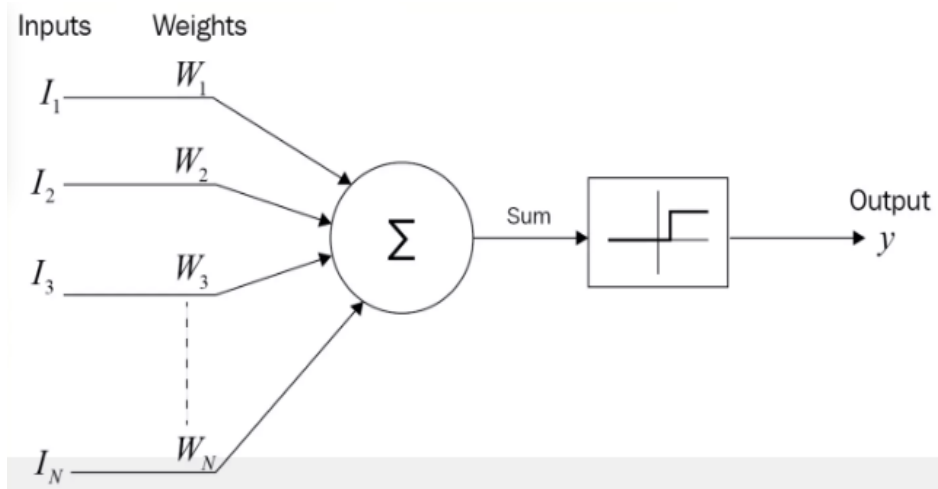


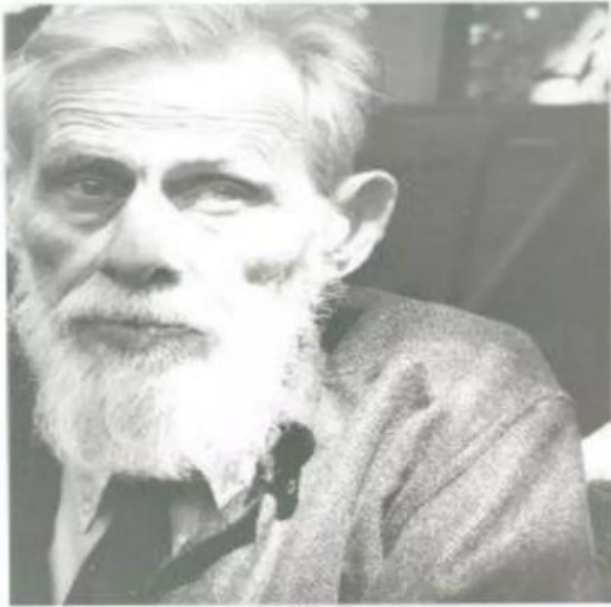
Neuron Anatomy



Part of Neuron	Description
Cell Body (Soma)	The core of the neuron containing the nucleus, Golgi body, endoplasmic reticulum, mitochondria, and other cellular components. It maintains the cell's functions, synthesizes proteins, and provides energy.
Dendrites	Branch-like structures that receive messages from other neurons and transmit them to the cell body. They are covered in synapses and increase the surface area for connections with other neurons.
Axon	A tube-like structure responsible for carrying electrical impulses away from the cell body to axon terminals. It can be covered by a myelin sheath, which insulates and speeds up the transmission of nerve impulses.
Axon Terminals	Found at the end of the axon, these specialized structures contain synapses and release neurotransmitter chemicals to communicate with other neurons.
Synapse	The junction between the axon terminal of one neuron and the dendrites of another neuron, where neurotransmitters are released and signals are transmitted from one neuron to another.
Myelin Sheath	A layer of fatty material that covers the axons of neurons, providing insulation between nerve cells and speeding up the conduction of nerve impulses.
Axon Hillock	A specialized structure located at the junction between the cell body and the axon. It integrates signals from multiple synapses and acts as a threshold detector, determining whether an action potential should be generated.

Mc Culloch Pitts Neuron





Warren S. McCulloch

The McCulloch-Pitts Neuron was the earliest neural network discovered in 1943.

The weights associated with communication links may be excitatory (weights are positive) or inhibitory (weights are negative).

A threshold activation function plays a major role in the M-P neuron. If the net input is greater than the sum of the input signals, the neuron fires.

Mostly used in the logic functions.

The activation of M-P neuron is always binary in nature.

It consists of a number of input units connected to a single output unit.

Salient features of McCulloch-Pitts artificial neuron

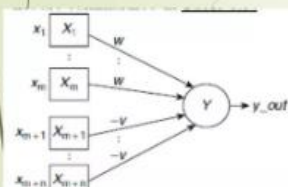


Fig. 6.11. Structure of a McCulloch-Pitts neuron

- 1 There are two kinds of input units, *excitatory*, and *inhibitory*. In Fig. 6.11 the excitatory inputs are shown as inputs $X_1, \dots, X_n$ and the inhibitory inputs are $X_{n+1}, \dots, X_{n+m}$. The excitatory inputs are connected to the output unit through positively weighted links. Inhibitory inputs have negative weights on their connecting paths to the output unit.
- 2 All excitatory weights have the same positive magnitude w and all inhibitory weights have the same negative magnitude $-v$.
- 3 The activation $y_{out} = f(y_{in})$ is binary, i.e., either 1 (in case the neuron fires), or 0 (in case the neuron does not fire).
- 4 The activation function is a binary step function. It is 1 if the net input y_{in} is greater than or equal to a given threshold value θ , and 0 otherwise.
- 5 The inhibition is absolute. A single inhibitory input should prevent the neuron from firing irrespective of the number of active excitatory inputs.

MCCULLOCH-PITTS NEURON

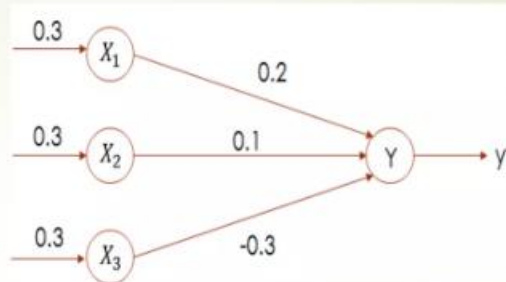
Activation function for MP neuron is

$$f(y) = \begin{cases} 1 & \text{if } y \geq \theta \\ 0 & \text{if } y < \theta \end{cases}$$

And

$$\theta > nw - p$$

Q1. For the network shown in Figure 1, calculate the weights are net input to the output neuron



Solution: The given neural net consists of three input neurons and one output neuron. The inputs and weights are

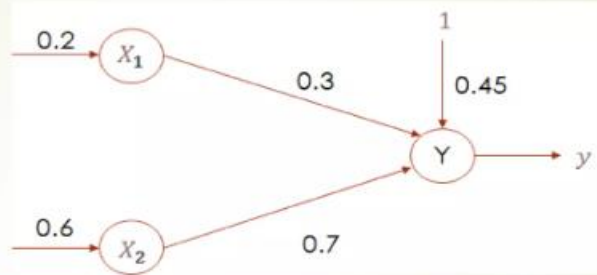
$$[x_1, x_2, x_3] = [0.3, 0.5, 0.6]$$

$$[w_1, w_2, w_3] = [0.2, 0.1, -0.3]$$

The net input can be calculated as

$$\begin{aligned} y_{in} &= x_1 w_1 + x_2 w_2 + x_3 w_3 \\ &= 0.3 \times 0.2 + 0.5 \times 0.1 + 0.6 \times (-0.3) \\ &= 0.06 + 0.05 - 0.18 = -0.07 \end{aligned}$$

Q2. Calculate the net input for the network shown in Figure 2 with bias included in the network

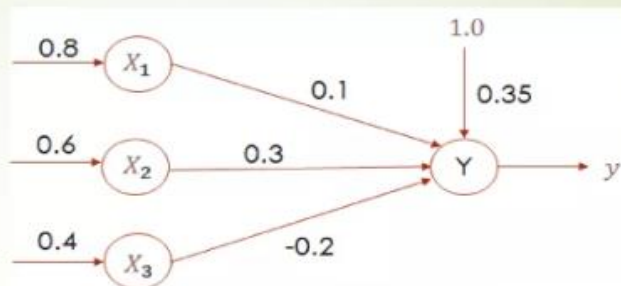


Solution: The given net consists of two input neurons a bias and an output neuron. The inputs are $[x_1, x_2] = [0.2, 0.6]$ and the weights are $[w_1, w_2] = [0.3, 0.7]$. Since the bias is included $b = 0.45$ and bias input x_0 is equal to 1. The net input is calculated as

$$\begin{aligned} y_{in} &= b + x_1 w_1 + x_2 w_2 \\ &= 0.45 + 0.2 \times 0.3 + 0.6 \times 0.7 \\ &= 0.45 + 0.06 + 0.42 = 0.93 \end{aligned}$$

Therefore, $y_{in} = 0.93$ is the net input.

Q3. Obtain the output of the neuron Y for the network shown in Figure 3 using activation function as: (i) binary sigmoidal and (ii) bipolar sigmoidal.



Solution: The given network has three neurons with bias and output neuron. These form a single layer network. The inputs are given as $[x_1, x_2, x_3] = [0.8, 0.6, 0.4]$ and the weights are $[w_1, w_2, w_3] = [0.1, 0.3, -0.2]$ with bias $b = 0.35$ (its input is always 1). The net input to the output neuron is

$$\begin{aligned} y_{in} &= b + \sum_{i=1}^n x_i w_i \\ [n=3, \text{ because only 3 input neurons are given}] \\ &= b + x_1 w_1 + x_2 w_2 + x_3 w_3 \\ &= 0.35 + 0.8 \times 0.1 + 0.6 \times 0.3 + 0.4 \times (-0.2) \\ &= 0.35 + 0.08 + 0.18 - 0.08 = 0.53 \end{aligned}$$

(i) For binary sigmoidal activation function,

$$y = f(y_{in}) = \frac{1}{1 + e^{-y_{in}}} = \frac{1}{1 + e^{-0.53}} = 0.625$$

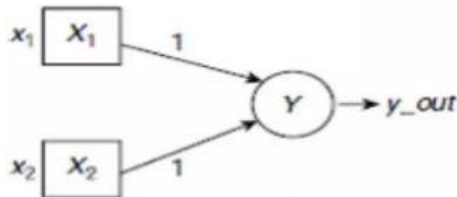
(ii) For bipolar sigmoidal activation function,

$$y = f(y_{in}) = \frac{2}{1 + e^{-y_{in}}} - 1 = \frac{2}{1 + e^{-0.53}} - 1 = 0.259$$

Example-4: M-P Neuron to implement the logical AND operation

(a) Truth Table

x_1	x_2	$x_1 \text{ AND } x_2$
0	0	0
0	1	0
1	0	0
1	1	1



(b) Neural structure

$$y_{in} = x_1 + x_2$$

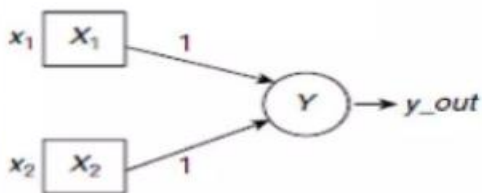
$$y_{out} = f(y_{in}) = \begin{cases} 1, & \text{if } y_{in} \geq 2, \\ 0, & \text{otherwise.} \end{cases}$$

(c) Activation function

Example-5: M-P Neuron to implement the logical OR operation

(a) Truth Table

x_1	x_2	$x_1 \text{ OR } x_2$
0	0	0
0	1	1
1	0	1
1	1	1



(b) Neural structure

$$y_{in} = x_1 + x_2$$

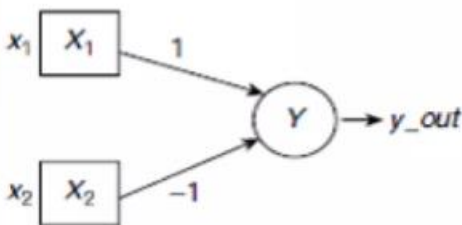
$$y_{out} = f(y_{in}) = \begin{cases} 1, & \text{if } y_{in} \geq 1, \\ 0, & \text{otherwise.} \end{cases}$$

(c) Activation function

Example-6: M-P Neuron to implement the logical AND-NOT operation

(a) Truth Table

x_1	x_2	$x_1 \text{ AND } (\text{NOT } x_2)$
0	0	0
0	1	0
1	0	1
1	1	0



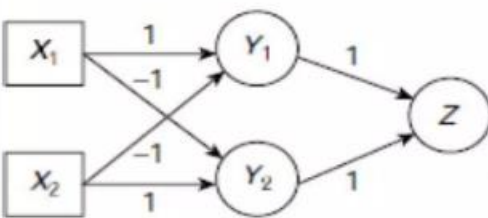
$$y_{in} = x_1 - x_2$$

$$y_{out} = f(y_{in}) = \begin{cases} 1, & \text{if } y_{in} \geq 1, \\ 0, & \text{otherwise.} \end{cases}$$

Example-7: M-P Neuron to implement the logical XOR operation

(a) Truth Table

x_1	x_2	$x_1 \text{ XOR } x_2$
0	0	0
0	1	1
1	0	1
1	1	0



$$f(\text{net}_{in}) = \begin{cases} 1, & \text{if } \text{net}_{in} \geq 1, \\ 0, & \text{otherwise.} \end{cases}$$

(b) Neural structure

(c) Activation function

Let's Take a real-world example:

A bank wants to decide if it can sanction a loan or not. There are 2 parameters to decide- Salary and Credit Score. So there can be 4 scenarios to assess-

1. High Salary and Good Credit Score
2. High Salary and Bad Credit Score

3. Low Salary and Good Credit Score

4. Low Salary and Bad Credit Score

Let $X_1 = 1$ denote high salary and $X_1 = 0$ denote Low salary and $X_2 = 1$ denote good credit score and $X_2 = 0$ denote bad credit score

Let the threshold value be 2. The truth table is as follows

X1	X2	$X_1 + X_2$	Loan approved
1	1	2	1
1	0	1	0
0	1	1	0
0	0	0	0

The truth table shows when the loan should be approved considering all the varying scenarios. In this case, the loan is approved only if the salary is high and the credit score is good. The McCulloch Pitt's model of neuron was mankind's first attempt at mimicking the human brain. And it was a fairly simple one too. It's no surprise it had many limitations-

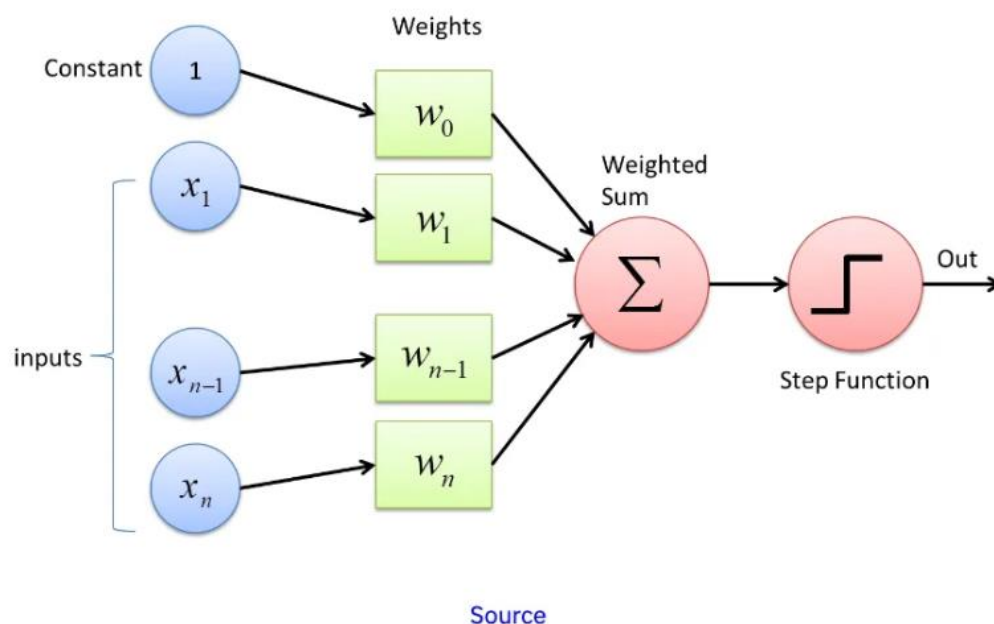
1. The model failed to capture and compute cases of non-binary inputs. It was limited by its ability to compute every case with 0 and 1 only.
2. The threshold had to be decided beforehand and needed manual computation instead of the model deciding itself.
3. Linearly separable functions couldn't be computed.

Perceptron

The **Perceptron** is one of the simplest **artificial neural network** architectures, introduced by Frank Rosenblatt in 1957. It is primarily used for **binary classification**.

At that time, traditional methods like Statistical Machine Learning and Conventional Programming were commonly used for predictions. Despite being one of the simplest forms of artificial neural networks, the Perceptron model proved to be highly effective in solving specific classification problems, laying the groundwork for advancements in AI and machine learning.

Perceptron is a type of [neural network](#) that performs binary classification that maps input features to an output decision, usually classifying data into one of two categories, such as 0 or 1.



Perceptron consists of a single layer of input nodes that are fully connected to a layer of output nodes. It is particularly good at learning **linearly separable patterns**. It utilizes a variation of artificial neurons called **Threshold Logic Units (TLU)**, which were first introduced by McCulloch and Walter Pitts in the 1940s. This foundational model has played a crucial role in the development of more advanced neural networks and machine learning algorithms.

Types of Perceptron

1. [Single-Layer Perceptron](#) is a type of perceptron is limited to learning linearly separable patterns. It is effective for tasks where the data can be divided into distinct categories through a straight line. While powerful in its simplicity, it struggles with more complex problems where the relationship between inputs and outputs is non-linear.

2. **Multi-Layer Perceptron** possess enhanced processing capabilities as they consist of two or more layers, adept at handling more complex patterns and relationships within the data.

Basic Components of Perceptron

A Perceptron is composed of key components that work together to process information and make predictions.

- **Input Features:** The perceptron takes multiple input features, each representing a characteristic of the input data.
- **Weights:** Each input feature is assigned a weight that determines its influence on the output. These weights are adjusted during training to find the optimal values.
- **Summation Function:** The perceptron calculates the weighted sum of its inputs, combining them with their respective weights.
- **Activation Function:** The weighted sum is passed through the **Heaviside step function**, comparing it to a threshold to produce a binary output (0 or 1).
- **Output:** The final output is determined by the activation function, often used for **binary classification** tasks.
- **Bias:** The bias term helps the perceptron make adjustments independent of the input, improving its flexibility in learning.

How does Perceptron work?

A weight is assigned to each input node of a perceptron, indicating the importance of that input in determining the output. The Perceptron's output is calculated as a weighted sum of the inputs, which is then passed through an activation function to decide whether the Perceptron will fire.

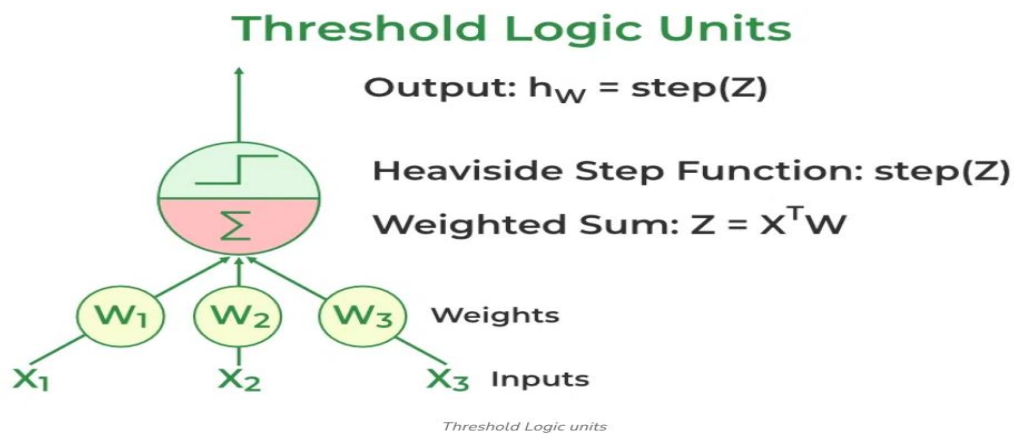
The weighted sum is computed as:

$$z = w_0 + w_1x_1 + w_2x_2 + \dots + w_{n-1}x_{n-1} + w_nx_n.$$

The step function compares this weighted sum to a threshold. If the input is larger than the threshold value, the output is 1; otherwise, it's 0. This is the most common activation function used in Perceptrons are represented by the Heaviside step function:

$$h(z) = \begin{cases} 0 & \text{if } z < \text{Threshold} \\ 1 & \text{if } z \geq \text{Threshold} \end{cases}$$

A perceptron consists of a single layer of Threshold Logic Units (TLU), with each TLU fully connected to all input nodes.



Example: Perceptron in Action

Let's take a simple example of classifying whether a given fruit is an apple or not based on two inputs: its weight (in grams) and its color (on a scale of 0 to 1, where 1 means red). The perceptron receives these inputs, multiplies them by their weights, adds a bias, and applies the activation function to decide whether the fruit is an apple or not.

- Input 1 (Weight): 150 grams
- Input 2 (Color): 0.9 (since the fruit is mostly red)
- Weights: [0.5, 1.0]
- Bias: 1.5

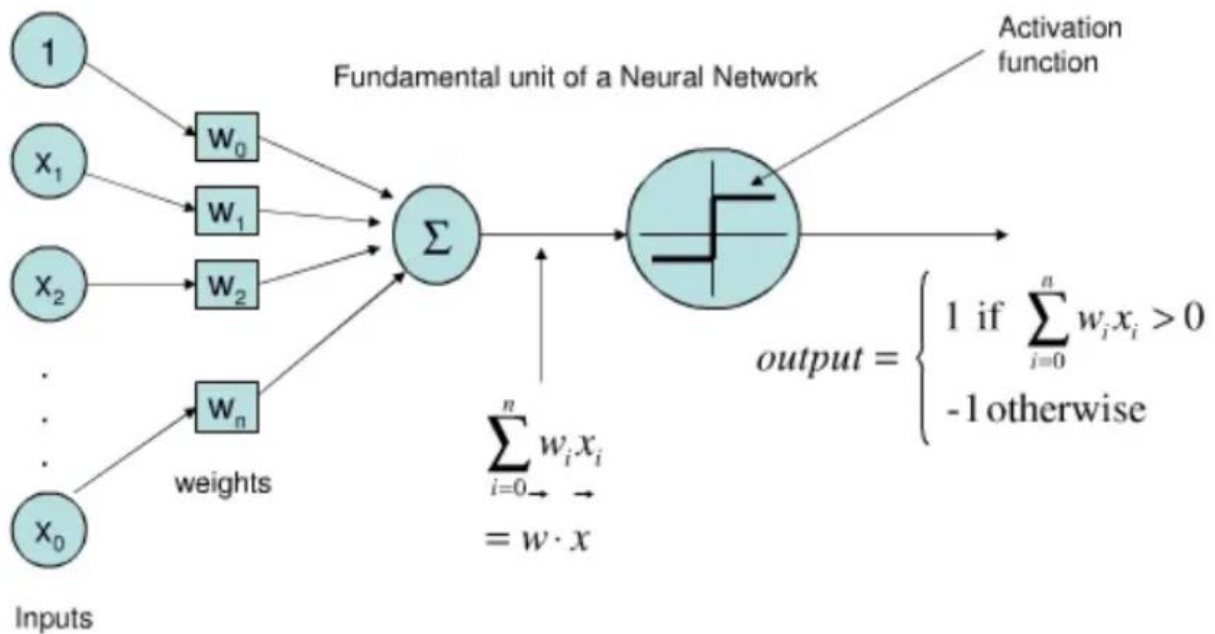
The perceptron's weighted sum would be:

$$(150 * 0.5) + (0.9 * 1.0) + 1.5 = 76.4$$

Let's assume the activation function uses a threshold of 75. Since $76.4 > 75$, the perceptron classifies the fruit as an apple (output = 1).

Linear Perceptron

A linear perceptron is a fundamental building block in deep learning, acting as a simple binary linear classifier. It's a foundational algorithm that learns a decision boundary to separate data into two classes by adjusting weights and bias based on training data. While it can only handle linearly separable data, it serves as a crucial concept for understanding more complex neural networks.



Perceptron Learning Algorithm

Perceptron Learning Algorithm is also understood as an Artificial Neuron or neural network unit that helps to detect certain input data computations in business intelligence. The perceptron learning algorithm is treated as the most straightforward Artificial Neural network. It is a supervised learning algorithm of binary classifiers. Hence, it is a single-layer neural network with four main parameters, **i.e.**, input values, weights and Bias, net sum, and an activation function.

What is the Perceptron Learning Algorithm?

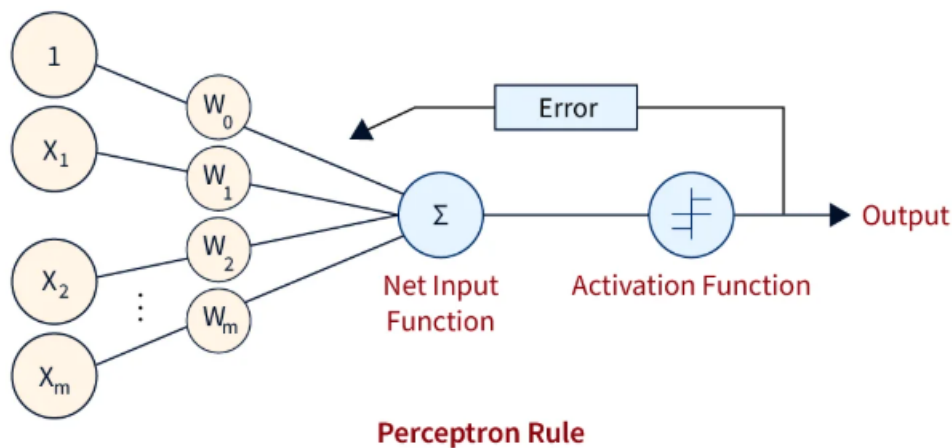
There are four significant steps in a perceptron learning algorithm:

1. First, multiply all input values with corresponding weight values and then add them to determine the weighted sum. Mathematically, we can calculate the weighted sum as follows:

$$\sum w_i * x_i = x_1 * w_1 + x_2 * w_2 + \dots + w_n * x_n$$

Add another essential term called bias 'b' to the weighted sum to improve the model performance. $\sum w_i * x_i + b$.

2. Next, an activation function is applied to this weighed sum, producing a binary or a continuous-valued output. $Y = f(\sum w_i * x_i + b)$
3. Next, the difference between this output and the actual target value is computed to get the error term, E, generally in terms of mean squared error. The steps up to this form the forward propagation part of the algorithm. $E = (Y - Y_{\text{actual}})^2$
4. We optimize this error (loss function) using an optimization algorithm. Generally, some form of gradient descent algorithm is used to find the optimal values of the hyperparameters like learning rate, weight, Bias, etc. This step forms the backward propagation part of the algorithm.



1. Binary Linear Classifier:

- The perceptron classifies data points into one of two categories (binary).
- It achieves this by finding a linear decision boundary (a line in 2D, a plane in 3D, or a hyperplane in higher dimensions) that separates the classes.
- This linear boundary is defined by the learned weights and bias.

2. How it Works:

- Inputs:

The perceptron receives multiple input values, each associated with a weight.

- Weighted Sum:

It calculates the weighted sum of these inputs, essentially multiplying each input by its corresponding weight and adding them up.

- Activation Function:

The weighted sum is then passed through an activation function, usually a step function (or sometimes a sigmoid in more advanced uses), which outputs either 0 or 1.

- Decision:

If the output is 1, the data point is classified into one category; if it's 0, it's classified into the other.

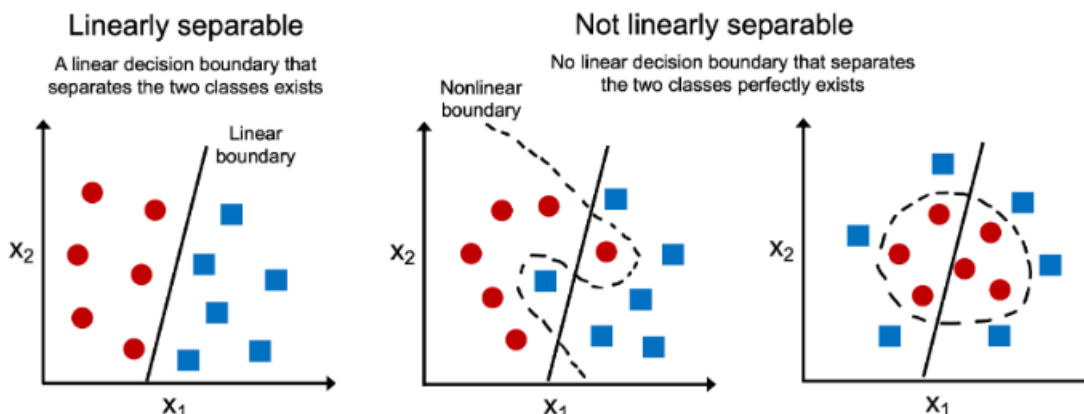
3. Learning Process:

- The perceptron learns by adjusting its weights based on the errors it makes during training.
- For each misclassified data point, the weights are updated to move the decision boundary slightly in a direction that reduces the error.
- This iterative process continues until the perceptron achieves a desired level of accuracy on the training data.

4. Limitations:

- Linear Separability: The perceptron can only learn linear decision boundaries.
- Non-linearly Separable Data: If the data cannot be separated by a straight line (or hyperplane), the perceptron will fail to learn an accurate classification.

linear separability describes a dataset where data points belonging to different classes can be clearly separated by a straight line (in 2D) or a hyperplane (in higher dimensions). Essentially, if you can draw a line (or hyperplane) that divides the data into distinct regions, each containing only one class, then the data is considered linearly separable.



5. Importance in Deep Learning:

- Despite its limitations, the perceptron is a fundamental building block for more complex neural networks.
- It introduces the core concepts of weighted inputs, activation functions, and learning through error correction, which are all essential in deep learning.
- Multilayer perceptrons (MLPs), which are composed of multiple layers of perceptrons, can overcome the limitation of linear separability and learn complex non-linear relationships in data.

Perceptron Convergence Theorem

Understanding Perceptron Convergence Theorem

The Perceptron Convergence Theorem is a fundamental result in the theory of neural networks. This theorem provides a guarantee about the performance of the perceptron algorithm under certain conditions.

Statement: The Perceptron Convergence Theorem states that if there exists a linear separation (a hyperplane) that can perfectly classify a given set of training examples, then the perceptron algorithm will converge to a solution that correctly classifies all the training examples.

Conditions for Convergence

For the perceptron algorithm to converge, the following conditions must be met:

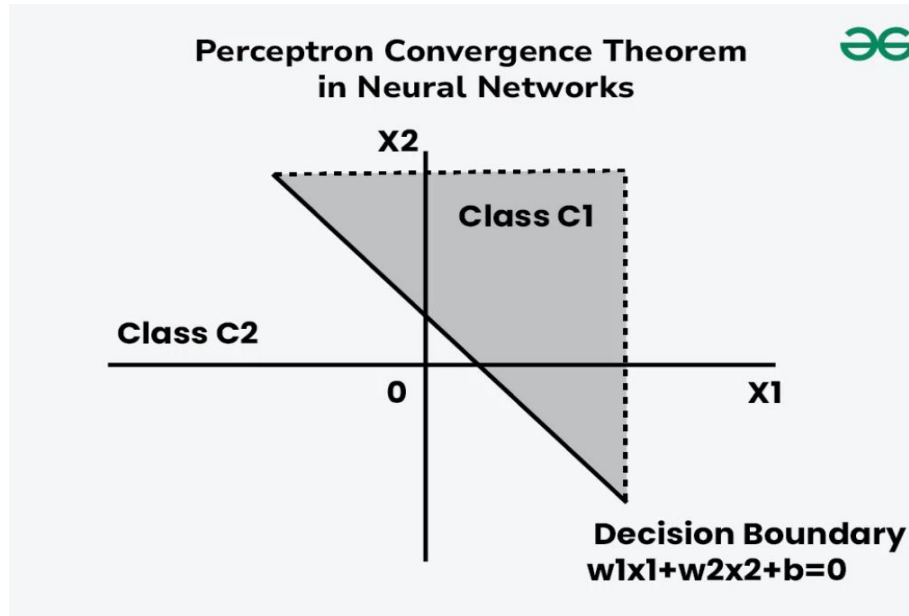
1. **Linear Separability:** The training data must be linearly separable. This means there exists a hyperplane that can separate the positive and negative examples without any errors.
2. **Non-zero Margin:** There must be a positive margin γ such that for all training examples (x_i, y_i) , the following condition holds:

$$y_i(w \cdot x_i + b) \geq \gamma$$

where w is the weight vector and b is the bias.

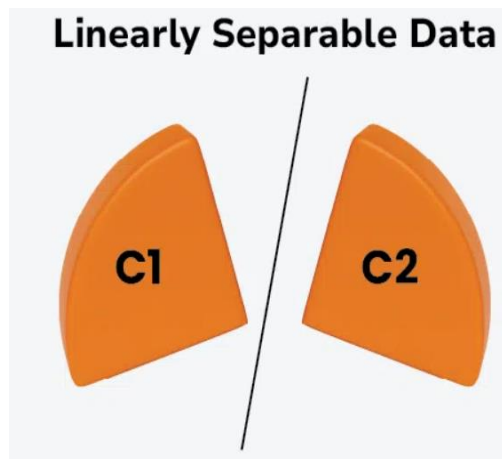
Implications for Theorem

- **Convergence:** For linearly separable data, the perceptron algorithm will always find a hyperplane that correctly classifies the training examples. For example- classes C1 and C2 can be classified by a hyperplane defined as:



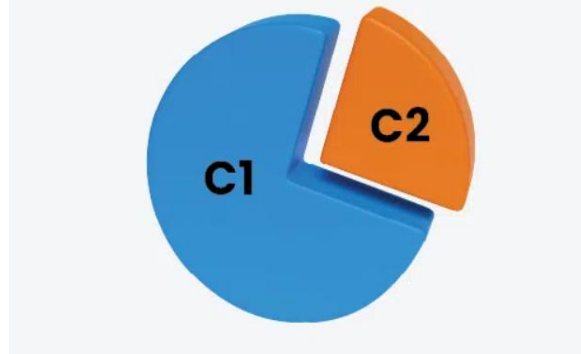
- **Separable Data:** If the data is not linearly separable, the perceptron algorithm will not converge and will continue to update its weights indefinitely.

1. Linearly Separable data: It refers to a set of data points can be perfectly divided into two distinct classes using a straight line (in two dimensions), a plane (in three dimensions), or a hyperplane (in higher dimensions). This means that there exists a linear boundary that can clearly separate all the data points of one class from those of the other class without any overlap.



3. **Non-linearly separable data:** It refers to a set of data points cannot be divided into two distinct classes using a straight line (in two dimensions), a plane (in three dimensions), or a hyperplane (in higher dimensions). In other words, no single linear boundary can perfectly separate the two classes.

Non- linearly separable data



Mathematical Formulation: Perceptron Convergence Theorem

The simple neuronal model is modified for perceptron convergence theorem. In order to do that, bias is treated as a synaptic weight driven by a fixed input which is equal to +1.

- Now input vector is defined as:

$$\circ x(n) = [+1, x_1(n), x_2(n), \dots, x_m(n)] \text{ where } n \text{ is the iteration step of the algorithm.}$$

- weight vector is defined as:

$$\circ w(n) = [b(n), w_1(n), w_2(n), \dots, w_m(n)]$$

- Linear combiner output $v(n)$ can be calculated as:

$$\circ v(n) = \sum_{i=1}^m w_i(n)x_i(n) = w^T(n)x(n)$$

For a fixed n , $w^T x = 0$ plotted in m -dimensional space defines a hyperplane as decision surface between two different classes.

Training:

Let $L1$ be the subset of training vectors which belongs to class $C1$ and $L2$ be the subset of training vector which belongs to class $C2$.

Classifier Training:

$L1$ and $L2$ will train the classifier if we adjust the weight vector w such that classes $C1$ and $C2$ are linearly separable:

- $w^T x > 0$ for $x \in C1$
- $w^T x \leq 0$ for $x \in C2$

Algorithm for updating weight vector is:

1. If n^{th} member $x(n)$ is correctly classified by $w(n)$ at the n^{th} iteration, then no correction is made.
 $w(n+1) = w(n)$
2. Otherwise, weight is updated as:

- $w(n+1) = w(n) - \eta x(n)$, if $w^T(n)x(n) > 0$ and $x(n) \in C2$
- $w(n+1) = w(n) + \eta x(n)$, if $w^T(n)x(n) \leq 0$ and $x(n) \in C1$

If η is constant, then we have a fixed increment adaptation rule.

If η is variable, then:

$$\eta(n)x^T(n)x(n) > |w^T(n)x(n)|$$

Workings of Perceptron Convergence Algorithm

Step-by-step working of this algorithm can be shown as:

1. Initialization: Start by setting the weights and bias to small random values. These weights determine the importance of each input feature.

2. Input: Feed an input data point (a vector of features) into the perceptron.

3. Weighted Sum: Calculate the weighted sum of the input features plus the bias. This is done by multiplying each input feature by its corresponding weight and adding them all up, along with the bias. $\text{weighted sum} = (w_1 \times x_1) + (w_2 \times x_2) + \dots + (w_n \times x_n) + b$

4. Activation Function: Apply the activation function to the weighted sum to get the output. In a simple perceptron, this is usually a step function that outputs 1 if the weighted sum is positive, and -1 (or 0) if it's negative.

5. Prediction: Compare the perceptron's output (prediction) to the actual label of the input data point (either 1 or -1).

6. Update Weights: If the prediction is incorrect, adjust the weights and bias. The weights are updated to reduce the error, moving the decision boundary closer to correctly classifying the input. The update rule is:

- $w_i = w_i + \Delta w_i$
- $b = b + \Delta b$

where

- $\Delta w_i = \eta * (y - y') * x_i$
- $\Delta b = \eta * (y - y')$
- η is learning rate parameter
- y is the actual label
- y' is the predicted label

7. Repeat: Repeat steps 2-6 for all data points in the training set. This process is called an epoch. Multiple epochs are performed until the perceptron correctly classifies all training data points or a maximum number of epochs is reached.

8. Convergence: If the data is linearly separable, the algorithm will eventually find the correct weights and bias to perfectly classify the training data. This means the perceptron has "converged."